World Cup Route Planner

Ryan Donnelly (rdonnelly49@qub.ac.uk)

Contents

Scope / Understanding of the challenge....3
Backend Tech choice (Node.js)....3
Backend Tasks....3
Task #1 - Cities....3
Task #2 – Matches....4
Task #3 - Route Optimizations....6
Task #4 - Unit Tests for Nearest Neighbor....8
Task #5 calculate cost and unit tests....8
Frontend challenge – RouteMap Component....11
Postman....13
ReadMe....13
Test Plan....13
Other notes....13
Bonus Challenge....14
Bonus Chall #1....14
Bonus chall #2....16
Future Improvement's....16

Scope / Understanding of the challenge

The purpose of this web app was to allow end users to optimize a travel itinerary for the FIFA World cup 2026 – which is going to be hosted between three countries; US, Mexica and Canada. System needed to incorporate game selection, route optimization and the budget calculation and comparison to the user allocated budget.

The idea of this project was to be able to add/modify pre existing code by forking the project from the junior-challenge template repo and to be able to have a working solution for several challenges whilst ensuring code quality was respected and to be able to have a understanding of the project. Also to write/develop meaningful tests and documentation.

Backend Tech choice (Node.js)

Node.js with express was chosen as one of the advantages was because of the event driven architecture makes it well-structured for handling concurrent API requests efficiently whilst this project is not heavily I/O bound at scale the model simplifies handling operations without introducing unnecessary complexity. As the frontend was developed using React + TypeScript which are both use JavaScript and by using Node.js the same core language is used throughout the entire application and makes sharing of types and data structures easier. If we compare it to Flash yes it is lightweight + flexible but it lacks the structure provided by TypeScript – and especially for this project having consistent data models for example city, route and match are important thus by going with Node.js provided better maintability. If we compare it to Spring boot which is great for enterprise level features and strong type safety, it adds more configuration overhead and for a project of this size Node.js is more appropriate. And .NET also suffers the same as Springboot being better suited to larger enterprise systems.

Backend Tasks

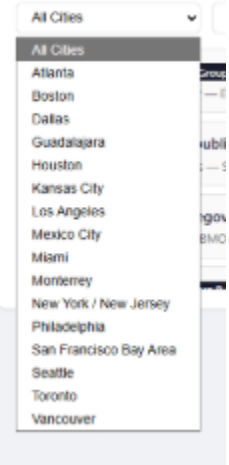
Task #1 - Cities

One of the key strengths of GET /api /cities is it s clear separation of concerns – the route handler is responsible for handling the HTTP request + response whilst the data get logic is delegated to the City Model – which improves maintability. Another positive is the Defensive programming practice – as the code validates that the data returned from the model layer is defined + in the expected array format before sending it to the client - which reduces the probability of returning runtimes. Also, the use of try-catch blocks ensures that unexpected errors are handled without crashing the server. Also, by returning appropriate HTTP status codes and error messages and avoids exposure to internal systems details in error responses are good practices.

This endpoint could be refactored further by adding a service layer as the route currently directly with the model layer so, if a layer was introduced this would help encapsulate business logic and keep this route cleaner. Also, an centralized error middleware could be used to capture/handle errors instead of them being duplicated in each route and ensure consistently throughout all endpoints.

But at the moment it is suffice especially for the size of the system.

```javascript
router.get('/', (_req, res) => {
  try {
    // Call the model layer to retrieve all cities
    const cities = CityModel.getAll();

    // Defensive check — ensure data exists and is an array
    if (!cities || !Array.isArray(cities)) {
      return res.status(500).json({
        error: 'Invalid data format returned from CityModel',
      });
    }

    // Return cities as JSON with default 200 status
    return res.status(200).json(cities);

  } catch (error) {
    // Log full error for debugging
    console.error('Error fetching cities:', error);

    // Return generic error message to client (avoid leaking internals)
    return res.status(500).json({
      error: 'Failed to fetch cities',
    });
  }
});
```



Task #2 – Matches

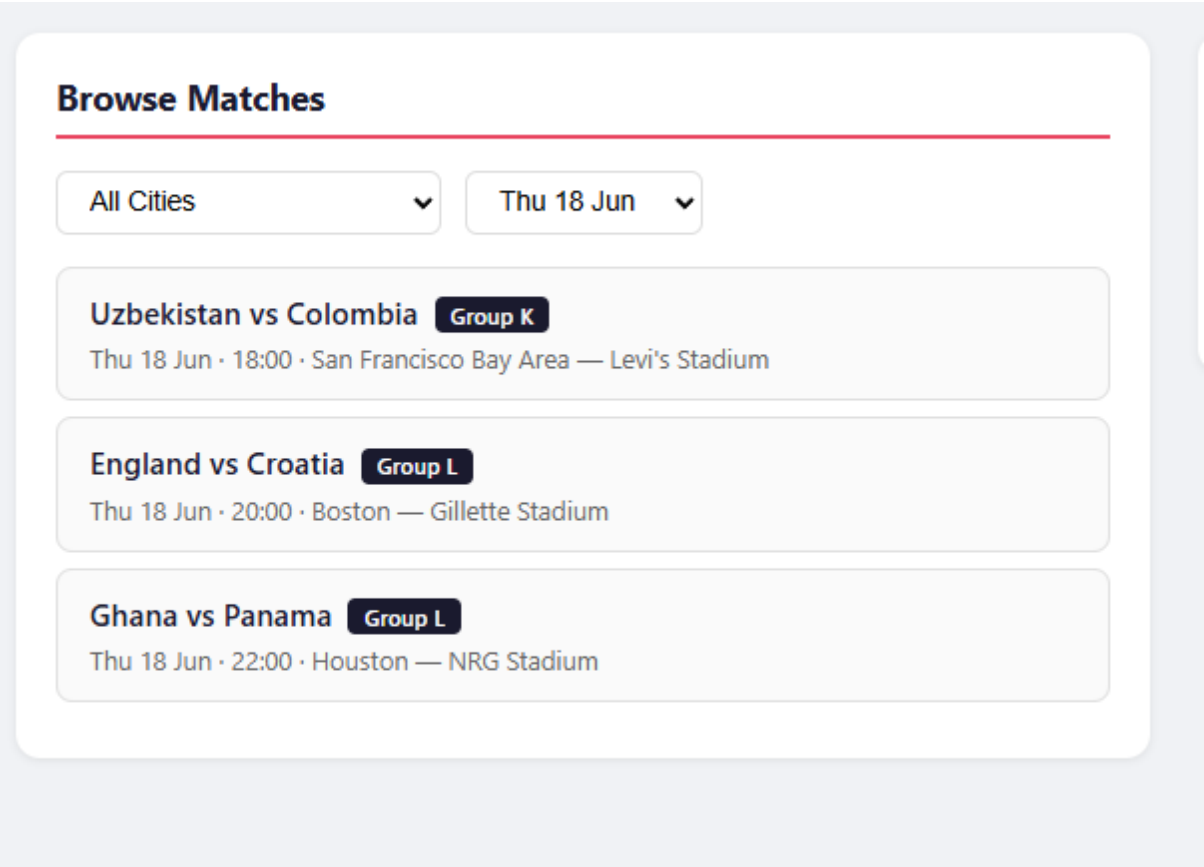
GET /api/matches and GET /api/matches/:id support both bulk data retrieval with filtering + individual resource access. GET /api/matches endpoint is flexible filtering capability – extracting optional query parameters city + date > passing them directly to the model layer the route is simpler whilst enabling powerful data querying = delegation (as the route handler focus on request handling whilst the model encapsulates the filtering logic thus improves code clarity + duplication.

Using type safe extraction of query parameters is a positive as by checking req.query + req.query. date are strings before using them thus avoiding runtime errors caused by unexpected types - which is important in JavaScript based environments where type safety is not reinforced at runtime.

Get /api/matches/:id endpoint retrieves a single resource using a path parameter which is good for RESTful conventions. – Includes appropriate validation to ensure an ID is provided and returns a 404-error code if the requested match does not existed again improving API reliability.

Again using try-catch blocks alongside errors are logged whilst returning a generic message to the client thus not exposing backend logic is good practice again.

These routes could be refactored by adding sorting parameters would improve performance and usability, especially when returning more matches, i.e if the system was converted to include many domestic football leagues with more matches.



```
router.get('/', (req, res) => {
  try {
    // Extract and safely type query params
    const city = typeof req.query.city === 'string' ? req.query.city : undefined;
    const date = typeof req.query.date === 'string' ? req.query.date : undefined;

    // Delegate filtering logic to model layer
    const matches = MatchModel.getAll({ city, date });

    return res.status(200).json(matches);

  } catch (error) {
    console.error('[GET /api/matches] Error:', error);

    return res.status(500).json({
      error: 'Failed to fetch matches',
    });
  }
});
```

```
router.get('/:id', (req, res) => {
  try {
    const { id } = req.params;

    if (!id) {
      return res.status(400).json({ error: 'Match ID is required' });
    }

    const match = MatchModel.getById(id);

    if (!match) {
      return res.status(404).json({ error: 'Match not found' });
    }

    return res.status(200).json(match);

  } catch (error) {
    console.error(`[GET /api/matches/${req.params.id}] Error:`, error);

    return res.status(500).json({
      error: 'Failed to fetch match',
    });
  }
});
```

Task #3 - Route Optimizations

POST /api/route/optimise/ endpoint is responsible for transforming a set of user selected matches into an optimized travel route which takes into account distance between cities. Strategy design pattern – encapsuulting the optimization logic within a NearestNeigbourStrategy class – route handler remains clean and focused soley on request validation + orchestration – design allows different optimization strategies such as DateOnlyStrategy to be swapped easily without modifying the endpoint logic – improves extensibility good OOP princes. Also enforces

validation of matchids cannot be null, orgincityid is provided and the orgin city exists and the matches are retrieved alongside appropriate HTTP status code and feedback to the client with internal logging to. The endpoint has a clear structure of validate the input, fetch required data from models and to return the result.

Alternative approaches to route optimization is a graph-based pathfinding if the problem was modelled as a graph with cities as nodes + flights as edges. However, this would require more complex model data and is more appropriate for real time routing system. Another approach would be TSP as it would evaluate all possible permutations of match orders to find the optimal routes. Whilst it would be the best solution the amount of time it would take would not be feasible if this project was ever expanded larger due to time constraints.

```
router.post('/optimise', (req, res) => {
  try {
    const { matchIds, originCityId } = req.body;

    // Validation
    if (!Array.isArray(matchIds) || matchIds.length === 0) {
      return res.status(400).json({ error: 'matchIds must be a non-empty array' });
    }

    if (!originCityId) {
      return res.status(400).json({ error: 'originCityId is required' });
    }


    // Fetch data
    const matches = MatchModel.getByIds(matchIds);
    const originCity = CityModel.getById(originCityId);

    if (!originCity) {
      return res.status(404).json({ error: 'Origin city not found' });
    }

    if (matches.length === 0) {
      return res.status(404).json({ error: 'No matches found for given IDs' });
    }

    const strategy = new NearestNeighbourStrategy();

    const route = strategy.optimise(matches, originCity);

    return res.status(200).json(route);

  } catch (error) {
    console.error('[POST /api/route/optimise] Error:', error);

    return res.status(500).json({
      error: 'Failed to optimise route',
    });
  }
});
```

Task #4 - Unit Tests for Nearest Neighbor

The tests focus on core functionality and key edge cases – ensuring that optimisaation logic behaves correctly.

The use of clear test structure (arrange, act and assert – each test explicitly sets up mock data – invokes the optimism method + verifies the expected output – improves readability + makes the intent of easy to understand. The use of mock data (cities + matches) is good as creating controlled test inputs – tests remain independ of external sources reducing the risk of contamination.

Happy test is valuable as it verifies multiple aspects of the algorithm at the one time.

Route contains expected number of stops
Total distance is greater than zero
Correct strategy name is returned
Route is marked as feasible
All required countries are included.

Inclusion of edge case tests strengths it – testing an empty matches array ensures te function handles of input gracefully and testing a single match verifies no unnecessary distance calculations are performed.

The use of beforeEach() to reset the strategy instance is also good practice, ensuring test isolation + preventing shared state between tests.

Additional tests could include missing countries scenario – test a route that does not include all required countries + verify that feasible is false + missingCountirs is populated. Another test could be minimum matches constraint – verify that routes with fewer than required number of matches are marked as not feasible. Another check could be distance accuracy check and another one for invalid data handling and to perform a test with a large dataset.

Task #5 calculate cost and unit tests

POST /api/route/budget handles cost calculation and to determine whether a user's selected users itinerary is feasible within a budget. Again this endpoint is well structured to follow input validation, get data from models, cost calculation (flights, tickets) and constraint validation and finally the response construction. Also demonstrates defensive programming as it validates that matched is a non-empty array, orgincity + budget are the correct type and the origin city exists and matches are successfully retrieved. Likewise HTTP status codes are retuned . The use of Map for flight lookup is efficient reducing the time complexity of retrieving flight prices from O(n) to O(1)- Also handles business logic of that the end user has to visit all required countries and determine whether the cost is within budget and generating suggestions when constrains are met. This endpoint could be refactored by handling multi accusation as the current implementation assumes one night per match which better approach would calculate nights based on the difference between match dates.

```
router.post('/budget', (req, res) => {
  try {
    const { matchIds, originCityId, budget } = req.body;
```

```
// Validation
if (!Array.isArray(matchIds) || matchIds.length === 0) {
  return res.status(400).json({ error: 'matchIds must be a non-empty array' });
}

if (typeof originCityId !== 'string' || typeof budget !== 'number') {
  return res.status(400).json({ error: 'originCityId and budget are required' });
}

const matches = MatchModel.getByIds(matchIds);
const originCity = CityModel.getById(originCityId);
const flights = FlightModel.getAll();

if (!originCity) {
  return res.status(404).json({ error: 'Origin city not found' });
}

if (matches.length === 0) {
  return res.status(404).json({ error: 'No matches found' });
}


// Build flight lookup
const flightMap = new Map<string, number>(
  flights.map((f: any) => [
    `${f.origin_city_id}-${f.destination_city_id}`,
    f.price_usd,
  ])
);


// Ticket cost
const ticketCost = matches.reduce((sum, m) => sum + m.ticketPrice, 0);


// Flight cost
let flightCost = 0;
let currentCityId = originCity.id;

for (const match of matches) {
  const key = `${currentCityId}-${match.city.id}`;
  const price = Number(flightMap.get(key) ?? 0);

  flightCost += price;
  currentCityId = match.city.id;
}

// Accommodation cost (1 night per match)
const accommodationCost = matches.reduce(
```

```
    (sum, m) => sum + m.city.accommodation_per_night,
    0
  );


  // Total cost
  const totalCost = ticketCost + flightCost + accommodationCost;

  // Country validation
  const countriesVisited = Array.from(
    new Set(matches.map(m => m.city.country))
  );

  const REQUIRED_COUNTRIES = ['USA', 'Mexico', 'Canada'];

  const missingCountries = REQUIRED_COUNTRIES.filter(
    c => !countriesVisited.includes(c)
  );

  // Feasibility
  const withinBudget = totalCost <= budget;
  const hasAllCountries = missingCountries.length === 0;
  const feasible = withinBudget && hasAllCountries;

  // Suggestions
  const suggestions: string[] = [];

  if (!withinBudget) {
    suggestions.push('Reduce number of matches or choose cheaper cities');
  }

  if (!hasAllCountries) {
    suggestions.push(`Add matches in: ${missingCountries.join(', ')}`);
  }

  // Response
 return res.status(200).json({
    feasible,
    countriesVisited,
    missingCountries,
    suggestions,

    costBreakdown: {
      flights: flightCost,
      accommodation: accommodationCost,
      tickets: ticketCost,
      total: totalCost,
    },
```

```
      minimumBudgetRequired: totalCost,
    });

  } catch (error) {
    console.error('[POST /api/route/budget] Error:', error);

    return res.status(500).json({
      error: 'Failed to calculate budget',
    });
  }
});
```

The unit tests written have a broad coverage of functional scenarios – tests include a happy path test > verifies correct behavior when the itinerary is valid. Test for budget constrains violations which ensure that the subsystem correctly flags infeasible routes and provides appropriate suggestions. Test for missing country coverage – to ensure that the constraint of all three countries of USA, Mexico, Canada are enforced. Validation tests for invalid input – ensuring robustness again malformed requests. Tests for error conditions e.g. origin city and/or invalid match id. And a correctness check for cost calculation – to verify that the total cost equals to the sum of its components. Also handling of edge cases strengths, the approach of handling missing flight routes, testing when total cost equals the budget validates boundary conditions and to verify a 404 response is returned ensures proper error handling for proper error handling for invalid resources. – By having clear test names, logical grouping using describe consistent assertion patterns and validation of both status code and response body structure.

Additional tests that could be added in later to improve coverage is to test negative budget input, duplicate match id's, zero budget, large input dataset and exact country coverage – test scenarios where exactly one match per required country is included to ensure constraints are minimally satisfied.

Frontend challenge – RouteMap Component

Key strength of this implementation is its handling of dynamic + potentially inconsistent structures – inclusion of the helper function to extract team names getTeamName ensures that both strings + object based of teams are supported – defensive programming approach reduces the risk of runtime errors and improves robustness of the component. The structure of the rendered output is also well designed with clearly labeled stop number, competing teams in a readable format, formatted kickoff date using locale aware formatting. The use of the provided CSS classes popup-match, popup-match-number and popup-match-date ensures visual consistency across the application to the design requirements. Another strength is the correct use of React list rendering practices – each rendered element includes a unique match.id which helps react update the dom and prevent potential rendering issues. Also good reability through the use of destructing (const {match, stopNumber } = stop) + logical separation of concerns within the mapping function – making the code easier to understand.

Potential improvements- extracting helper function is currently defined inside the render loop and is recreated on every render – moving the function outside the component would improve performance and code hygiene and to have reusable date formatting.

An alternative design would involve pre processing the route data before rendering – this would simplify rendering logic + reduce repeated computations or another approach would be a dedicated child component for each match entry – would improve separation of concerns + make the UI more scalable as additional features such as icons.

Despite these alternatives/refactoring the current implementation is suited to the scope of the project as it achieves good balance between simplicity + functionality – avoiding unnecessary abstraction whilst marinating clarity and correctness.

```
        stops.map((stop) => {
  const { match, stopNumber } = stop;

  const getTeamName = (team: any) =>
    typeof team === 'string' ? team : team.name;

  const homeTeam = getTeamName(match.homeTeam);
  const awayTeam = getTeamName(match.awayTeam);

  const formattedDate = new Date(match.kickoff).toLocaleDateString('en-GB', {
    day: 'numeric',
    month: 'short',
  });

  return (
    <div key={match.id} className="popup-match">
      <span className="popup-match-number">Stop {stopNumber}</span>
      <div>
        {homeTeam} vs {awayTeam}
      </div>
      <span className="popup-match-date">{formattedDate}</span>
    </div>
  );
})

}
```

The unit tests implemented using Jest + React testing library focus on validating core component behavior by mocking the react-leaflet library – tests avoid reliance on external map rendering + concentrate on verifying logical output – ensures that placeholder is correctly displayed when no route is provided, map container renders when valid route data exists, markers are generated based on number of stops, edge case such as empty routes are handled well. However, in the future I would like to combine these unit/integration tests with E2E tests

using Cypress as this would be able to validate the full user workflow and UI just not the API and component logic in isolation. For example to test map is rendering, markers are rendered with stop number, team names, kickoff date, and the user flow test of user chooses matches, choose origin city, plan route and calculate budget and that the UI updates at each stage and alongside ensuring that appropriate messages appear when requirements are not met.

Postman

Postman was useful as it quickly allowed me to test the endpoints after I implemented the endpoint without having to write any unit tests however, it is more time consuming to test edge cases thus I just tested the core functionality and left further testing to the unit tests.

ReadMe

I created a new READMe.md for the root of this project which gives a project overview, detailed architecture of the frontend/backend as long as RESTY APIs. = A detailed folder structure for the backend/frontend which has been generated by JavaScript scripts. There is also setup instruct ions and API endpoint documentation. There is also a section on testing and description of the documentation available.

Having a README in a GitHub Repo is beneficial as it improves developer onboarding i.e say if an invite an collaborator to this project or any of my other repo's it will be easier for them to understand the purpose of the project and quicken the process of allowing them to start the project start committing.

Test Plan

Unit Test results have been documented within the unit tests folder.

	A	B	C	D	E	F	G
1	Test ID	Test Description	Input / Setup	Expected Result	Actual Result	Status	
2	1	Render placeholder when route is null	route = null, originCity = null	Placeholder message "Validate a route..." is displayed	Placeholder message rendered correctly	Pass	

Other notes

One thing I thought was ambiguous was that the user could not set the budget manually in the frontend so, I have adjusted the code to give the user more flexibility of individual travel route instead of having one universal budget as one size does not fit all.

```
{/* Budget & Origin Header */}
<div className="trip-settings-header">
  <div className="trip-settings-content">
    <div className="trip-setting">
      <label htmlFor="budget">Your Budget</label>
      <div className="budget-input-wrapper-header">
        <span className="currency-symbol-header">$</span>
     <input
        id="budget"          Chat (CTRL + I) / Share (CTRL + L)
      type="number"
      value={budget}
      onChange={(e) => setBudget(Number(e.target.value))}
      className="budget-input"
        />
      </div>
    </div>
```

Bonus Challenge

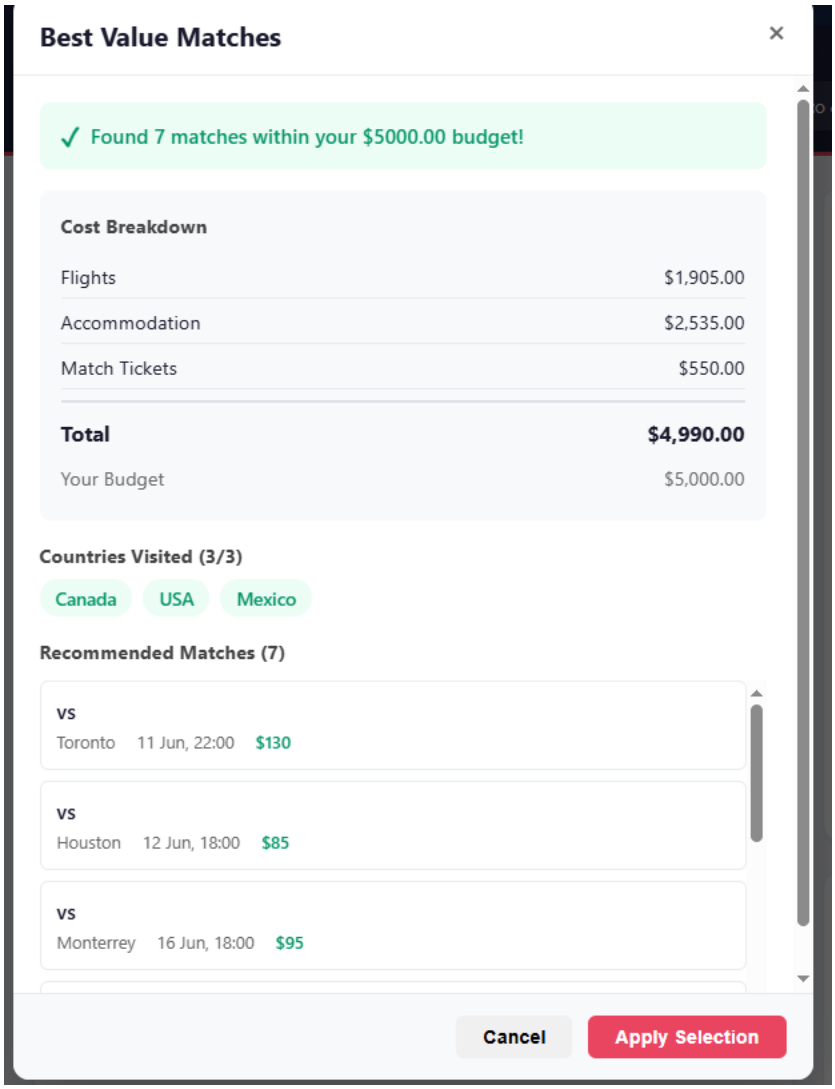
Bonus Chall #1

POST /api/route/best-value is respoble for identifying the best comboination of matches within a given budget - one of the most advanced features in the system. Clear separation of concerns – major strength is the delegation of business logic to the findBestValue function – ensures the route handler remains clean and focues on request/response handling – compex optimisation logic is encapsulated in a dedicated module and the code is easier to test + maintain.

An alternative approach could be something like Greedy pre filtering which filters out very expensive matches and then sort matches by ticket price or async processing where run optimisation asynchronously and polls results.

```
router.post('/best-value', (req, res) => {
  try {
    const { budget, originCityId } = req.body;

    const matches = MatchModel.getAllWithCities(); // important
    const originCity = CityModel.getById(originCityId);
    const flights = FlightModel.getAll();

    if (!originCity) {
      return res.status(404).json({ error: 'Origin city not found' });
    }

    const result = findBestValue(
      matches,
      budget,
      originCityId,
      flights,
      originCity
    );

    console.log('BEST VALUE RESULT:', result);

    return res.status(200).json(result);

  } catch (error) {
    console.error(error);
    return res.status(500).json({ error: 'Failed to find best value' });
  }
});
```

Bonus chall #2



Future Improvement's

Future improvements that could be implemented is to allow users to select how many countries they wish to visit, choose specific countries of interest and get a break down of that countries matches/cities and customise constraints such as minimum matches or travel preferences e.g. car/train or flight – This would make the application more user-centric + adaptable to real world scenarios rather than strictly following predefined rules.

Also to improve UI/UX validation – at present certain UI behaviours may cause confusion when actions are available but not valid for example buttons calculate cost or route plan may appear clickable even when requirements are not met – future improvement would disable invalid actions until conditions are satisfied and provide clear feedback messages explaining why an action is unavailable – would improve usability + reduce user confusion.

CI/CD pipeline integration would be a big win as tests are run manually and locally – introducing CI/CD pipeline would significantly improve development workflow and would help the case of

“my tests work on my machine but not another developers machine” (if ci/cd tests passes we are more confident that the tests are correct. Improvements would be automatically running jest and frontend tests automatically every commit (if changes are made to the core code) tests will be ran ands will be notified if any tests break due to changes, prevents merges if tests fail and automating builds + deployment.

Database and deployment – as the project is currently is uses a lightweight database setup – for production level – migrating to a robust database such as Firebase or PostgreSQL, containiersing the application using Docker, deploying us orchestrations tools e.g. Rancher or Kubernetes – would improve stability and ease of deployment. I would also included more integration tests to more edge cases, performance testing for detailed optimization algorithms and stress testing API endpoints under load.

I would also like to remove the other backends of python, springboot and .net to reduce the size of the project as these backends are unused.